

Academia

Student & Course Management System

Project Report

Course	Programming in Java
Submission Type	Build Your Own Project (VITyarthi)
Submitted By	Manish
Date	September 09, 2026

2. Introduction

This report documents **Academia**, a console-based Java application built for the "Programming in Java" course project. The project applies core object-oriented programming concepts — classes, interfaces, custom exceptions, collections, file I/O, and a layered package structure — to a realistic problem: managing students, courses, enrollments, and grades for a small academic institute. All data is persisted to CSV files, and the entire application runs from a terminal with no external database or GUI dependency.

3. Problem Statement

Small academic departments that don't run a full ERP system still need to track the same core facts: which students exist, which courses are offered, who is enrolled in what, and what grade each student earned. Managing this in spreadsheets is error-prone — duplicate enrollments slip through, records get overwritten, and there is no single place to compute a student's GPA. Academia addresses this at a small scale: a single-user CLI tool that stores records in CSV files, validates every operation, and can generate a per-student transcript with GPA or a GPA distribution across all students — without needing a database server.

4. Functional Requirements

The system is organized into three major functional modules, each with a clear input/output structure:

- **Student Management** — add, list, search, update, and delete student records (input: registration no., name, email; output: a generated Student ID such as STU0001).
- **Course Management** — add, list, search, update, and delete course records (input: code, title, credits, instructor, semester; output: confirmation and a listable record).
- **Enrollment, Grades & Reporting** — enroll/unenroll a student in a course, record a letter grade, and generate a transcript (with computed GPA) or a GPA distribution report across all students (input: student ID, course code, grade; output: transcript text, GPA bands).

The user interacts with the system through a numbered CLI menu at every level: a main menu routes to one of the three modules (plus Backup and Reports), each of which shows its own sub-menu of numbered actions, prompts for the relevant fields, and prints a confirmation or error message before returning to the main menu.

5. Non-Functional Requirements

Requirement	How Academia addresses it
Performance	All operations run in-memory against small in-process lists, with CSV writes only on change — every
Reliability	Every mutating action (add/update/delete/enroll/grade) immediately persists to its CSV file, so data is
Usability	A numbered, always-visible menu at every level; invalid numeric input is caught and re-prompted inst
Maintainability	Code is split into model / service / exception / util packages, so a change to storage format only touch
Error handling strategy	Three custom checked exceptions turn invalid operations into a clear, caught message rather than an

Resource efficiency	The dataset lives in memory as plain Java objects with small CSV files; no persistent network connections
---------------------	---

6. System Architecture

Academia follows a layered architecture: a presentation layer (Main.java) handles the CLI menu and user I/O; a service layer contains all business logic (StudentService, CourseService, EnrollmentService, ReportService); a model layer holds plain data classes (Student, Course, Enrollment, Instructor); a utility layer provides cross-cutting concerns (CSV I/O, ID generation, two generic interfaces); and a persistence layer of CSV files under data/. Each layer only depends on the layer(s) beneath it, which keeps the CLI free of business logic and the business logic free of file-format details.

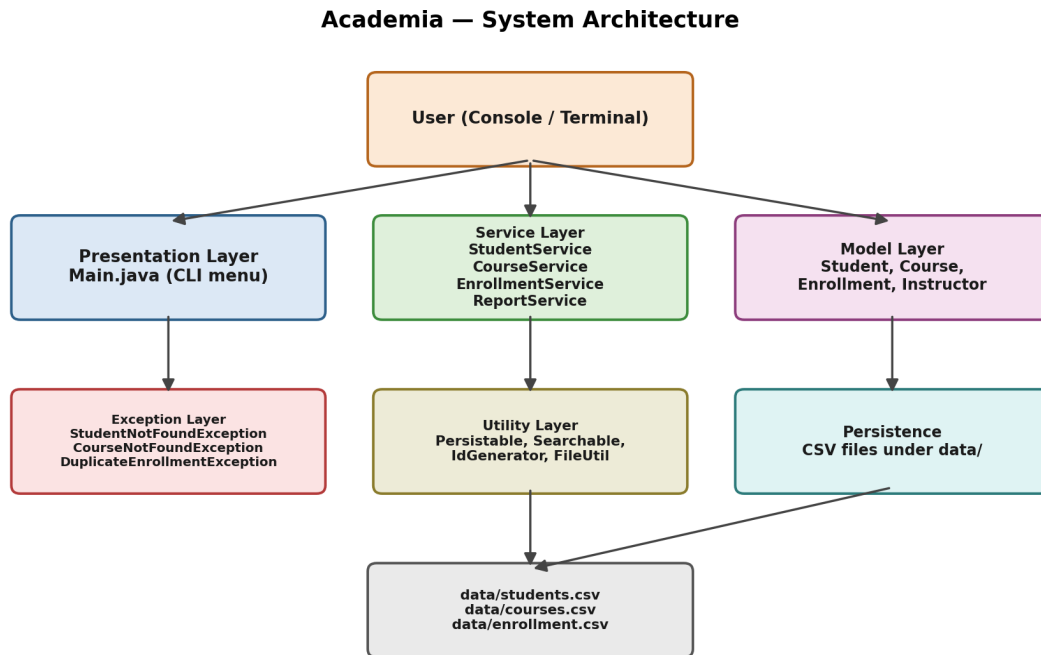


Figure 1: System architecture — presentation, service, model, utility, and persistence layers.

7. Design Diagrams

7.1 Use Case Diagram

Academia — Use Case Diagram

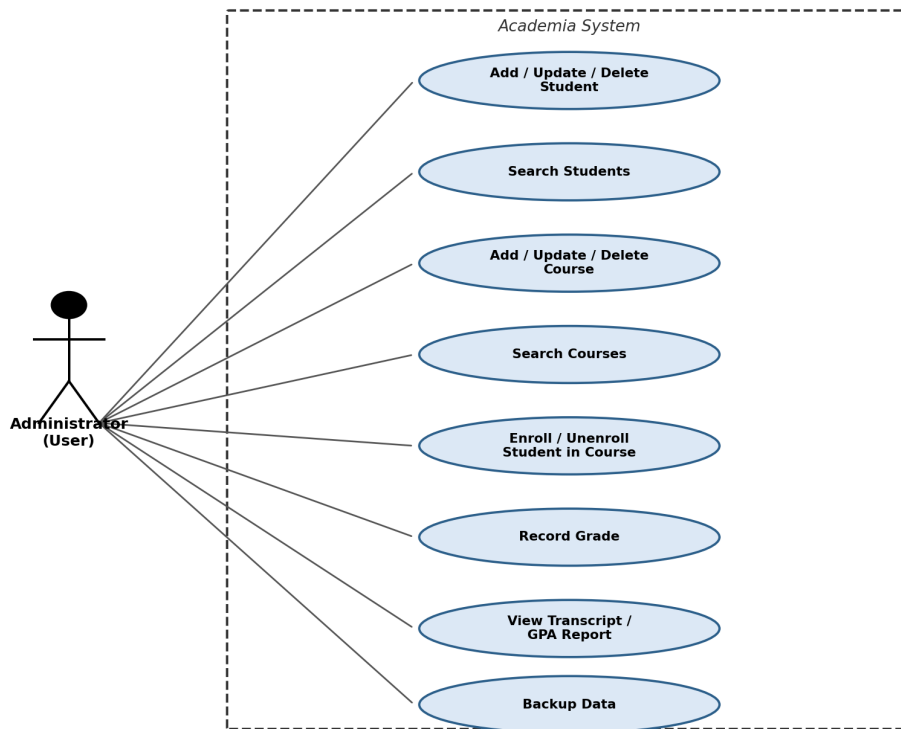


Figure 2: Use case diagram — the Administrator interacts with all eight core use cases.

7.2 Process Flow / Workflow Diagram

Academia — Process Flow / Workflow

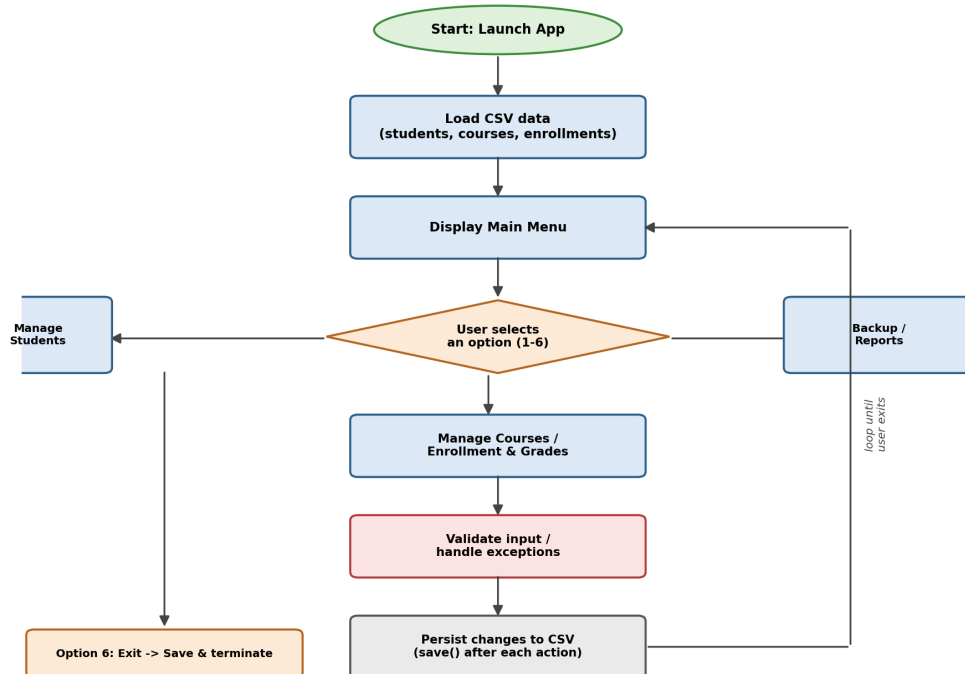


Figure 3: Process flow — the main loop from launch through menu dispatch to persistence.

7.3 Sequence Diagram — Enroll a Student

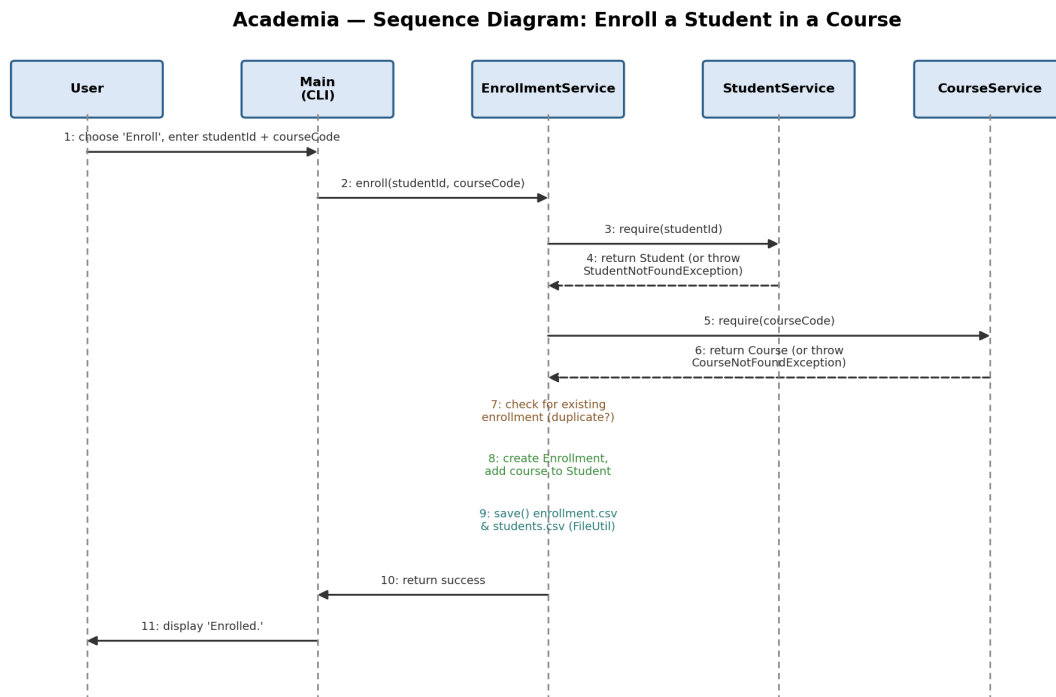


Figure 4: Sequence diagram for the enroll-a-student flow, including both failure paths.

7.4 Class Diagram

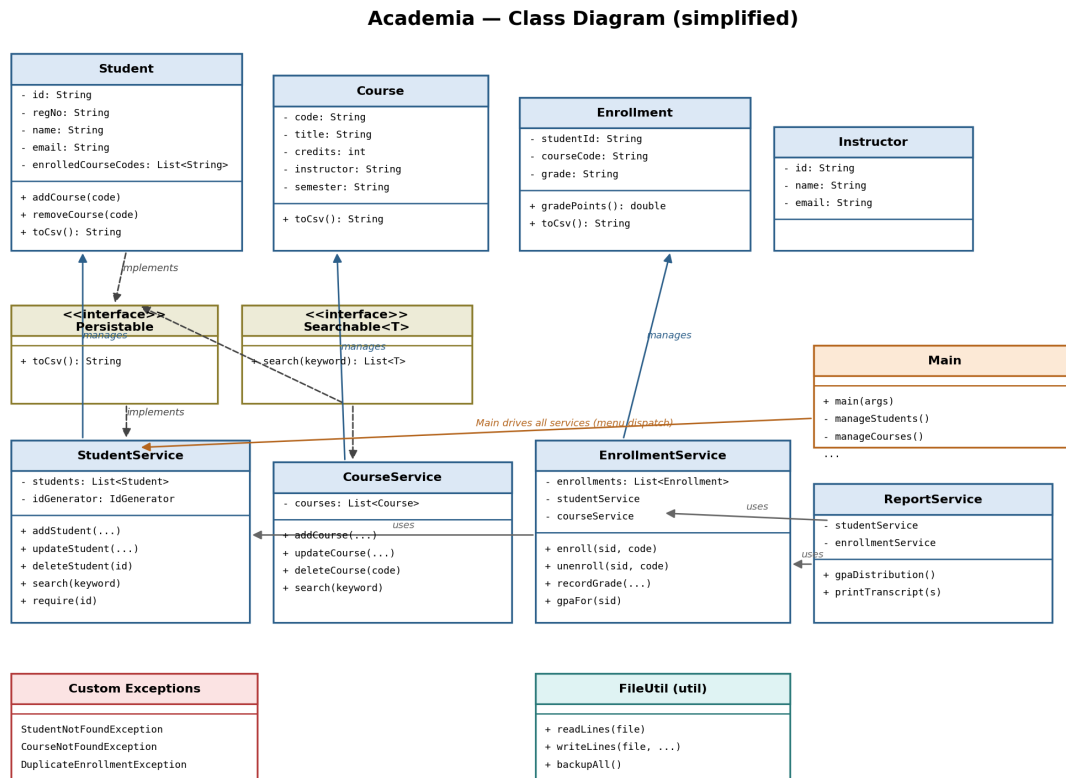


Figure 5: Simplified class diagram showing models, interfaces, services, and Main.

Note on storage design: Academia deliberately uses flat CSV files rather than a relational database, per the project brief's freedom to choose an appropriate storage approach for a Java CLI project. There is therefore

no ER diagram; the equivalent "schema" is the three CSV column layouts documented in Section 9 (Implementation Details) and in the README.

8. Design Decisions & Rationale

Why CSV instead of a database

CSV files keep the project runnable with zero setup — no JDBC driver, no server, no credentials — which matches the "fully executable via the command line" requirement and keeps the project's dependency footprint at just the JDK.

Why a layered package structure

Separating model / service / exception / util means the CLI (Main.java) never touches a file path directly, and the business logic never touches System.in/out directly. This made it straightforward to unit-test logic mentally by tracing one layer at a time, and would make swapping storage or adding a GUI later a localized change.

Why custom checked exceptions

StudentNotFoundException, CourseNotFoundException, and DuplicateEnrollmentException are checked exceptions so the compiler forces every call site in Main to handle the failure case explicitly, instead of relying on the caller remembering to check a return value or catch an unchecked RuntimeException.

Why two generic interfaces (Persistable, Searchable<T>)

Every model class needs to become a CSV line, and every service needs keyword search. Making these interfaces rather than duplicating the logic per class keeps the contract explicit and lets new entity types plug into the same save/search pattern.

Why immediate persistence over an explicit "Save" action

Saving after every mutating action (rather than only on exit) means a crash or forced-quit never loses more than the single in-flight action — a small reliability trade-off in favor of extra disk writes that are cheap at this data scale.

9. Implementation Details

The project has 16 Java files across four packages (model, service, exception, util) plus the Main entry point. Key implementation points:

- **CSV schema** — students.csv: id, regNo, name, email, courses (pipe-separated course codes). courses.csv: code, title, credits, instructor, semester. enrollment.csv: studentId, courseCode, grade.
- **ID generation** — IdGenerator produces sequential IDs like STU0001, and re-seeds itself from the highest existing ID on load so restarting the app never reuses or collides with an ID.
- **GPA calculation** — Enrollment.gradePoints() maps letter grades (S/A/B/C/D/E/F) to a 10-point scale; EnrollmentService.gpaFor() averages grade points across only the graded enrollments for a student, returning -1 ("not available") when no grades exist yet.
- **Backup** — FileUtil.backupAll() copies every file currently in data/ into a new data/backup_<yyyyMMdd_HHmmss>/ folder using java.nio.file.Files.copy with REPLACE_EXISTING, so repeated backups in the same second don't collide (the timestamp has second-level resolution) and existing backups are never partially overwritten.

Representative code — GPA calculation (Enrollment.java):

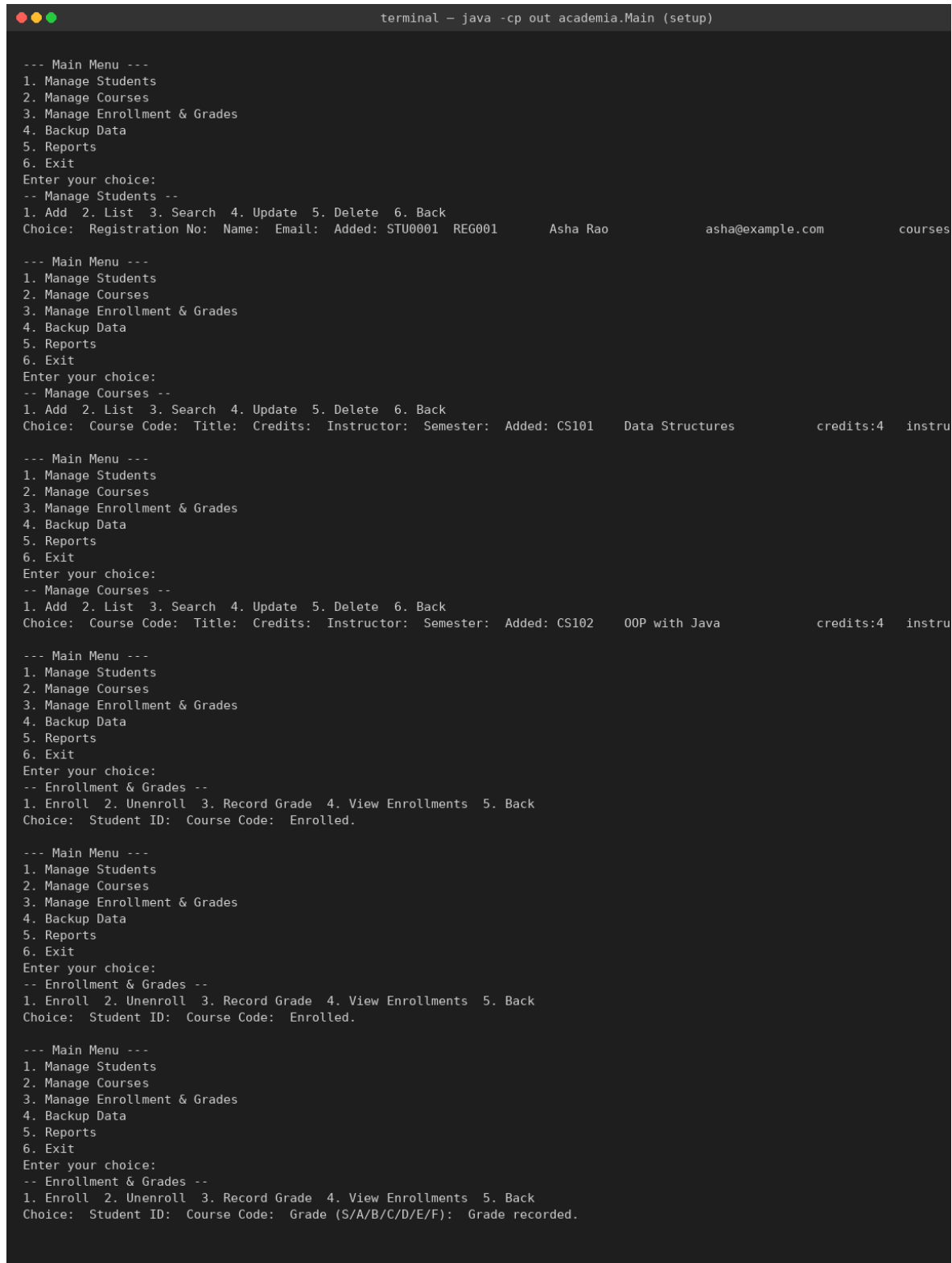
```
public double gradePoints() {
    if (grade == null || grade.isBlank()) return -1;
    return switch (grade.toUpperCase()) {
        case "S" -> 10.0;
        case "A" -> 9.0;
        case "B" -> 8.0;
        case "C" -> 7.0;
        case "D" -> 6.0;
        case "E" -> 5.0;
        default -> 0.0; // "F"
    };
}
```

Representative code — duplicate-enrollment check (EnrollmentService.java):

```
public Enrollment enroll(String studentId, String courseCode)
    throws StudentNotFoundException, CourseNotFoundException, DuplicateEnrollmentException {
    Student student = studentService.require(studentId);
    courseService.require(courseCode);
    boolean already = enrollments.stream()
        .anyMatch(e -> e.getStudentId().equals(studentId)
            && e.getCourseCode().equals(courseCode));
    if (already) {
        throw new DuplicateEnrollmentException(studentId, courseCode);
    }
    Enrollment e = new Enrollment(studentId, courseCode, "");
    enrollments.add(e);
    student.addCourse(courseCode);
    save();
    studentService.save();
    return e;
}
```

10. Screenshots / Results

The following screenshots show one continuous session: adding a student and two courses, enrolling the student in both, and recording grades.



```
terminal - java -cp out academia.Main (setup)

--- Main Menu ---
1. Manage Students
2. Manage Courses
3. Manage Enrollment & Grades
4. Backup Data
5. Reports
6. Exit
Enter your choice:
-- Manage Students --
1. Add 2. List 3. Search 4. Update 5. Delete 6. Back
Choice: Registration No: Name: Email: Added: STU0001 REG001      Asha Rao      asha@example.com      courses

--- Main Menu ---
1. Manage Students
2. Manage Courses
3. Manage Enrollment & Grades
4. Backup Data
5. Reports
6. Exit
Enter your choice:
-- Manage Courses --
1. Add 2. List 3. Search 4. Update 5. Delete 6. Back
Choice: Course Code: Title: Credits: Instructor: Semester: Added: CS101      Data Structures      credits:4      instru

--- Main Menu ---
1. Manage Students
2. Manage Courses
3. Manage Enrollment & Grades
4. Backup Data
5. Reports
6. Exit
Enter your choice:
-- Enrollment & Grades --
1. Enroll 2. Unenroll 3. Record Grade 4. View Enrollments 5. Back
Choice: Student ID: Course Code: Enrolled.

--- Main Menu ---
1. Manage Students
2. Manage Courses
3. Manage Enrollment & Grades
4. Backup Data
5. Reports
6. Exit
Enter your choice:
-- Enrollment & Grades --
1. Enroll 2. Unenroll 3. Record Grade 4. View Enrollments 5. Back
Choice: Student ID: Course Code: Enrolled.

--- Main Menu ---
1. Manage Students
2. Manage Courses
3. Manage Enrollment & Grades
4. Backup Data
5. Reports
6. Exit
Enter your choice:
-- Enrollment & Grades --
1. Enroll 2. Unenroll 3. Record Grade 4. View Enrollments 5. Back
Choice: Student ID: Course Code: Grade (S/A/B/C/D/E/F): Grade recorded.
```

Figure 6: Adding a student, two courses, enrolling, and recording a grade.

Continuing the same session: viewing enrollments, printing a transcript with computed GPA, the GPA distribution report, and a backup.

```
terminal — java -cp out academia.Main (reports)

--- Main Menu ---
1. Manage Students
2. Manage Courses
3. Manage Enrollment & Grades
4. Backup Data
5. Reports
6. Exit
Enter your choice:
-- Enrollment & Grades --
1. Enroll 2. Unenroll 3. Record Grade 4. View Enrollments 5. Back
Choice: Student ID: Course Code: Grade (S/A/B/C/D/E/F): Grade recorded.

--- Main Menu ---
1. Manage Students
2. Manage Courses
3. Manage Enrollment & Grades
4. Backup Data
5. Reports
6. Exit
Enter your choice:
-- Enrollment & Grades --
1. Enroll 2. Unenroll 3. Record Grade 4. View Enrollments 5. Back
Choice: STU0001 -> CS101 grade:A
STU0001 -> CS102 grade:S

--- Main Menu ---
1. Manage Students
2. Manage Courses
3. Manage Enrollment & Grades
4. Backup Data
5. Reports
6. Exit
Enter your choice:
-- Reports --
1. GPA Distribution 2. Student Transcript 3. Back
Choice: Student ID: Transcript for Asha Rao (STU0001)
-----
STU0001 -> CS101 grade:A
STU0001 -> CS102 grade:S
-----
GPA: 9.50

--- Main Menu ---
1. Manage Students
2. Manage Courses
3. Manage Enrollment & Grades
4. Backup Data
5. Reports
6. Exit
Enter your choice:
-- Reports --
1. GPA Distribution 2. Student Transcript 3. Back
Choice: 9.0 - 10.0 : 1
7.5 - 8.9 : 0
6.0 - 7.4 : 0
Below 6.0 : 0
No grades yet : 0

--- Main Menu ---
1. Manage Students
2. Manage Courses
3. Manage Enrollment & Grades
4. Backup Data
5. Reports
6. Exit
Enter your choice: Backup created at: /home/claude/academia_project/data/backup_20260909_073959

--- Main Menu ---
1. Manage Students
2. Manage Courses
3. Manage Enrollment & Grades
4. Backup Data
5. Reports
6. Exit
Enter your choice: Data saved. Goodbye!
```

Figure 7: Transcript with GPA, GPA distribution report, and backup confirmation.

11. Testing Approach

As a CRUD-style CLI application, testing was performed as structured manual validation covering both the happy path and the exception paths for each module, rather than an automated unit-test suite:

- Add → List → Search → Update → Delete for both students and courses, confirming each step reflects the previous one.
- Enroll a valid student/course pair, then attempt to enroll the same pair again — confirmed `DuplicateEnrollmentException` produces a clear message instead of a duplicate row.
- Attempt to enroll a nonexistent student ID and a nonexistent course code — confirmed `StudentNotFoundException` / `CourseNotFoundException` are thrown and caught with a readable message, not an uncaught stack trace.
- Record grades and verify GPA computation against a hand-calculated expected value.
- Exit and relaunch the application — confirmed all students, courses, and enrollments reload exactly as they were, and new IDs continue from the correct sequence.
- Run Backup Data and confirm a new timestamped folder appears containing accurate copies of all three CSV files.

12. Challenges Faced

ID collisions after restart

Initially, IDs restarted from STU0001 on every launch. This was fixed by having `IdGenerator` scan existing records on load and seed its counter to one past the highest existing ID.

Representing "no grade yet" vs. a real grade

An empty string and a null grade needed to be treated identically so an ungraded enrollment doesn't silently count as an F in GPA calculations — `gradePoints()` explicitly checks for both and returns a sentinel -1 to mean "not gradable yet", which the reporting layer filters out before averaging.

Keeping students.csv and enrollment.csv consistent

A student's enrolled-course list is stored redundantly on both the Student record and the Enrollment records for fast lookups in either direction. Enroll/unenroll had to be made to update and save both files together so they can't drift out of sync.

13. Learnings & Key Takeaways

Building Academia reinforced how much a small amount of upfront structure (separating model, service, exception, and util packages) pays off even in a project this size — it made it possible to change how data is stored without touching the CLI, and to add new report types without touching the enrollment logic. Using checked exceptions instead of print-and-continue error handling also made every failure path visible at compile time rather than something that only shows up during testing. Finally, working through the GPA "no grade yet" edge case was a good reminder that sentinel values (like -1 for "not available") need to be handled consistently at every layer that consumes them, not just where they're produced.

14. Future Enhancements

- Swap CSV storage for an embedded database (e.g. SQLite via JDBC) behind the same FileUtil-like interface, without changing the service layer's public API.
- Support multiple grade components per course (assignments, midterm, final) instead of a single letter grade per enrollment.
- Add a simple authentication layer so multiple administrators can use the system with an audit trail of who made each change.
- Add automated unit tests (JUnit) for the service layer, especially GPA calculation and the duplicate-enrollment check.
- Export transcripts and reports to PDF directly from the application.

15. References

- Oracle. *The Java Tutorials*. <https://docs.oracle.com/javase/tutorial/>
- Oracle. *java.nio.file.Files API documentation*.
<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/nio/file/Files.html>
- VITyarthi. *Build Your Own Project — General Project Instructions & Submission Guidelines*. (course-provided brief).